Звіт з заочної практичної роботи №1

Дисципліна: Безпека інформаційних систем
Студент: [Впишіть ваше Прізвище та Ім'я]
Група: [Впишіть вашу групу, напр. КН-31]
Репозиторій GitHub: [Вставте посилання на ваш репозиторій]
Останній коміт: [Хеш коміту, напр. a1b2c3d]

1. Опис проєкту

Розроблено захищений REST API системи «Електронний деканат». Реалізовано багаторівневий захист (Defense in Depth): ізольоване контейнерне середовище (Docker + SQLite), модель контролю доступу RBAC (Role-Based Access Control), хешування паролів (bcrypt), токенна автентифікація (JWT access/refresh) та захист на рівні HTTP (Security Headers, CORS, Rate Limiting, санітизація XSS).

2. Інструкція із запуску

Для розгортання проєкту з чистого клону виконайте наступні команди у терміналі:

```
git clone [посилання_на_ваш_репозиторій] cd information-security-practice cp .env.example .env # Налаштуйте JWT_SECRET_KEY у файлі .env docker compose up -d --build docker compose exec api alembic upgrade head docker compose exec api python -m app.seed
```

Адреси після запуску:

- API: `http://localhost:8000`
- Документація Swagger UI: `http://localhost:8000/docs`

3. Тестові облікові записи

Логін (username)	Роль	Дозволи (Permissions)
admin	admin	Усі дозволи (read_grades, edit_grades, read_schedule, manage_users, manage_groups, manage_subjects, view_reports)
teacher01	teacher	read_grades, edit_grades, read_schedule, view_reports
student01	student	read_grades, read_schedule

Примітка: Паролі зберігаються у БД виключно у вигляді bcrypt-хешів.

4. Маршрути (Ендпоінти) та доступ

Маршрут	Метод	Необхідна роль / Доступ	Опис
`/health`	GET	Public (без токена)	Статус системи та кількість таблиць БД
`/auth/register`	POST	Public (Rate limit: 10/хв)	Реєстрація з валідацією пароля та імені
`/auth/login`	POST	Public (Rate limit: 5/хв)	Отримання JWT токенів (захист від brute force)
`/auth/me`	GET	Будь-яка (потрібен токен)	Отримання інформації про поточного користувача
`/students/me/grades`	GET	**student**	Перегляд власних оцінок студентом
`/teachers/grades`	POST	**teacher, admin**	Виставлення оцінок із санітизацією коментарів
`/admin/users`	GET	**admin**	Перегляд списку всіх користувачів

5. Моделювання загроз (STRIDE)

Категорія загроз (STRIDE)	Контрзахід у коді	Що можна покращити в майбутньому
Spoofing (Підробка)	JWT токени з підписом HS256. CORS з жорстким allow-list (CORS_ORIGINS).	Перехід на асиметричний підпис RS256. Зберігання JWT у HttpOnly Cookie замість LocalStorage.
Tampering (Втручання)	Санітизація XSS у коментарях через `bleach`. Перевірка `full_name` через allow-list (RegEx).	Впровадження повноцінного WAF (Web Application Firewall) на рівні reverse proxy (Nginx).
Repudiation (Відмова)	Поле `assigned_by` в оцінках (підзвітність дій). Користувач визначається з JWT токена на сервері.	Ведення централізованих логів аудиту (Audit Logs) усіх дій з оцінками у захищеному сховищі.
Information Disclosure	Заголовки безпеки (CSP, X-Frame-Options, nosniff, HSTS). Уніфіковані помилки 401 при логіні.	Шифрування БД у стані спокою. Приховування детальних stack traces у продакшені.
DoS (Відмова в обслуговуванні)	Rate Limiting через `slowapi` (5 запитів/хв на логін). Pydantic обмеження довжини строк.	Використання Redis для Rate Limiting замість пам'яті. Обмеження розміру payload на рівні сервера.
Elevation of Privilege	Механізм RBAC (ролі з БД). Токен перевіряється на тип (access/refresh).	Автоматичне відкликання сесій при зміні прав (Redis-blacklist для JWT).

6. Фінальна перевірка системи

Усі перевірки виконані з використанням `curl` після ініціалізації бази даних.

№	Сценарій / Запит	Очікуваний результат	Фактичний результат (Код)
1	GET /health	200, tables: 8	200 OK
2	POST /auth/register (валідні дані)	201, без password_hash	201 Created
3	POST /auth/register (дублікат username)	409 Conflict	409 Conflict
4	POST /auth/register ("password": "password")	422 (слабкий пароль)	422 Unprocessable Entity
5	POST /auth/register ("full_name": "<script>")	422 (недопустимі символи)	422 Unprocessable Entity
6	POST /auth/login (admin)	200, access/refresh токени	200 OK
7	POST /auth/login (невірний пароль)	401 "Невірний логін або пароль"	401 Unauthorized
8	GET /auth/me (без токена)	401	401 Unauthorized
9	GET /auth/me (підроблений токен)	401	401 Unauthorized
10	GET /auth/me (Refresh замість access)	401	401 Unauthorized
11	POST /auth/refresh	200, нова пара токенів	200 OK
12	GET /students/me/grades (Студент)	200, свої оцінки	200 OK
13	GET /admin/users (Студент)	403 Forbidden	403 Forbidden
14	GET /admin/users (Викладач)	403 Forbidden	403 Forbidden
15	GET /admin/users (Адмін)	200, список користувачів	200 OK
16	POST /teachers/grades ("grade": 9999)	422 (поза діапазоном 0-100)	422 Unprocessable Entity
17	POST /teachers/grades (comment зі <script>)	201, тег вирізано санітайзером	201 Created
18	GET /health (заголовки)	Наявні CSP, HSTS, nosniff	Усі заголовки безпеки присутні
19	OPTIONS /auth/login (Origin: evil.example)	400, без allow-origin	400 Bad Request
20	Rate Limiting (6 логінів за хвилину)	429 Too Many Requests	429 Too Many Requests

** Скріншоти виконання (Swagger UI, перевірка заголовків та 429 статусу) додано до папки `docs/screenshots/` у репозиторії.*